IEMS 469 Assignment 2: Deep Q-Network

Jianing Chen · GitHub: jianingchenNU · Repository:
https://github.com/jianingchenNU/mlds_assignment2_Jianing

1. How to run

```
pip install -r requirements.txt          # plus torch from the PyTorch index, see README
python train.py --env cartpole --name cartpole_main --seed 2
python train.py --env pacman --name pacman_main --seed 0 \
    --episodes 5000 --max_hours 20 --lr_final 5e-5
python evaluate.py --run runs/cartpole_main --episodes 500
python evaluate.py --run runs/pacman_main --episodes 500 --ckpt last.pt --device cuda
python evaluate.py --run runs/pacman_main --random --episodes 100
python plots.py --run runs/<name>          # cartpole_main, pacman_main
python extra_plots.py --all
python validate/t1_gamma_sweep.py        # same pattern for t2 ... t7
```

The README lists every flag, the resume procedure and the per-run output files.

2. Implementation

The Q-function is a neural network trained from a replay buffer against a hard-copied target network. The target is the Double DQN target $y = r + \gamma(1 - \text{done}) \cdot Q_{\text{target}}(s', \text{argmax}_a\, Q_{\text{online}}(s', a))$, with Huber loss and gradient-norm clipping at 10. `done` is the environment's `terminated` flag only. A time-limit truncation ends the episode but is still bootstrapped. MsPacman frames go through the assignment's preprocessing function and are stacked 4 deep. The buffer stores each 88×80 int8 frame once and rebuilds stacks at sample time. Hyperparameters follow Mnih et al. (2015), with Adam, Double DQN targets and a 300k-frame buffer due to memory. CartPole ran on the CPU and MsPacman on a single RTX 5070 Ti (`config.json` records the device name).

Table 1. Hyperparameters (from each run's config.json) and run size (from train.csv).

	CartPole-v1	**ALE/MsPacman-v5**
γ	0.95	0.99
Network	MLP 4-128-128-2	Conv 32×8×8/4, 64×4×4/2, 64×3×3/1, FC 512, 9 outputs
Input	4-dim state	4 × 88×80 int8 frames, scaled by 1/128
Optimizer	Adam, lr 5×10^{-4}	Adam, lr $10^{-4} \rightarrow 5\times10^{-5}$ (linear), eps 1.5×10^{-4}
Buffer / batch	50,000 / 64	300,000 frames / 32
Warmup / train every	1,000 steps / 1 step	50,000 steps / 4 steps
Target sync	every 500 env steps	every 10,000 env steps
ε schedule	1.0 → 0.02 over 10k steps	1.0 → 0.1 over 1M, → 0.01 over the next 1M
Learning reward	r	sign(r); lost life = terminal for the target
Episodes / env steps / wall time	500 / 62,909 / 72 s	5,000 / 3,453,304 / 2.14 h
Device	CPU	RTX 5070 Ti Laptop GPU

3. CartPole

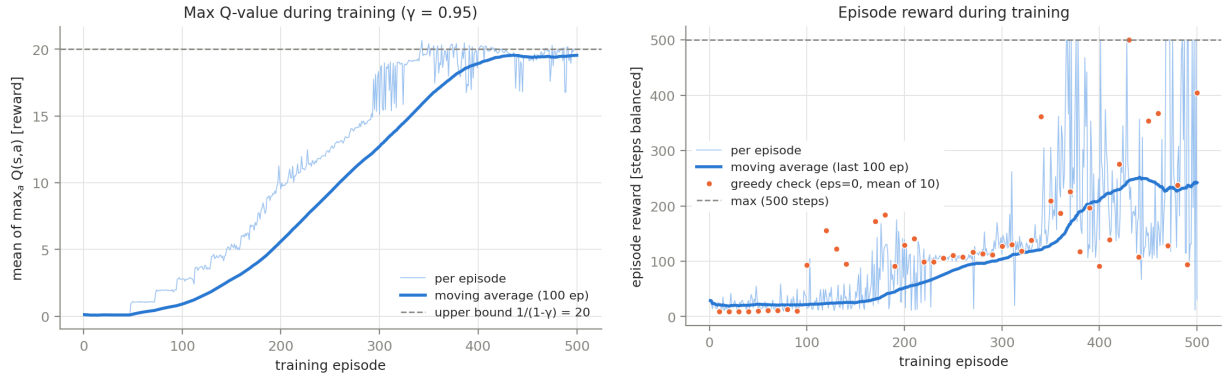


Figure 1. CartPole training. (a) Mean of $\max_a$ Q(s,a) per episode with its 100-episode moving average. (b) Episode reward with MA100 and greedy checks (ε = 0, mean of 10 episodes).

In Fig. 1a, the 100-episode average of max Q reaches 19.55 at episode 500. The mean over the last 50 episodes is 19.50, against the bound $1/(1-\gamma) = 20$. Early on, the per-episode curve rises in steps at episodes 47, 73, 94 and 113. These are the episodes where the step count crosses multiples of 500, the target sync period. Episodes then last about 20 steps, so one sync covers 19-26 episodes. Between episodes 40 and 200, the mean jump in Q is 0.43 at sync episodes and 0.16 at other episodes.

In Fig. 1b, MA100 rises from 21.8 (episode 100) to 103.5 (episode 300) and 242.1 (episode 500). MA100 stays far below 500 because ε stays at 0.02 from episode 232 on and the network keeps updating every step. 21 of the 500 training episodes reach 500 steps. The greedy check reached a mean of 500 at episode 430, which saved `best.pt`.

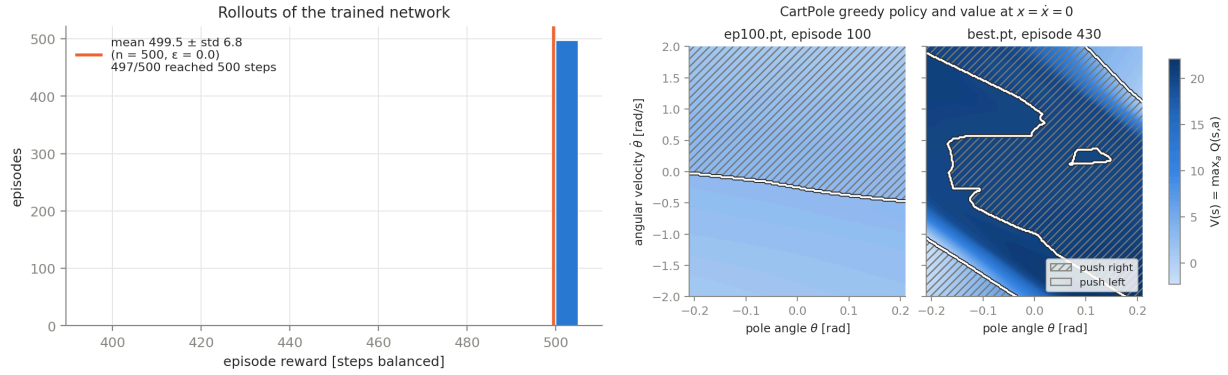


Figure 2. (a) 500 evaluation rollouts of best.pt (ε = 0), 5-step bins. (b) Greedy action (hatched = push right) and V(s) = $\max_a$ Q over pole angle θ and angular velocity $\dot{\theta}$ at $x = \dot{x} = 0$, episode-100 snapshot vs best.pt (episode 430).

In Fig. 2a, the mean reward is 499.5 with std 6.8 over 500 episodes. 497 of 500 episodes reach the 500-step limit, and the minimum is 399.

In Fig. 2b, at episode 100 the decision boundary is close to $\dot{\theta} = 0$, so the action depends almost only on angular velocity, and V stays near the low end of the scale. At episode 430 the boundary runs diagonally in $(\theta, \dot{\theta})$, so the push direction depends on both angle and velocity. V is close to 20 across the central band of states the policy can recover from.

4. MsPacman

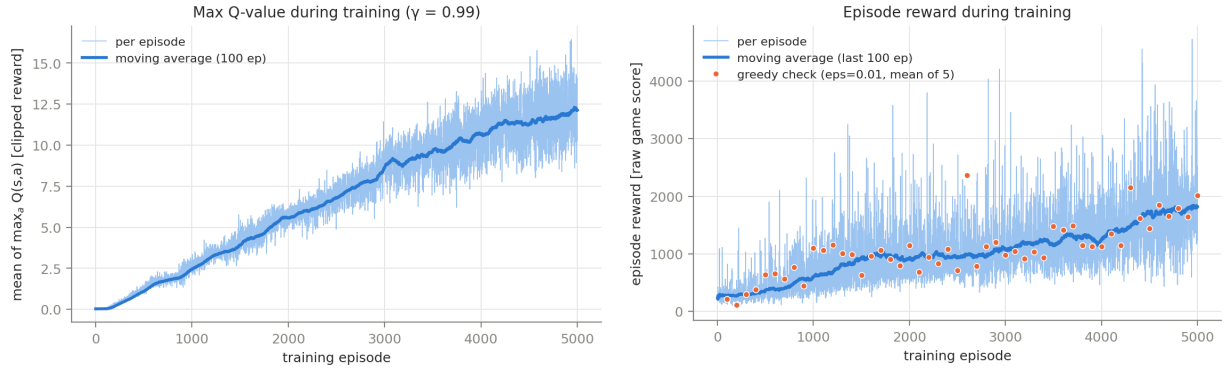


Figure 3. MsPacman training. (a) Mean of $\max_a$ Q(s,a) per episode (clipped-reward units) with MA100. (b) Episode reward (raw game score) with MA100 and greedy checks (ε = 0.01, mean of 5 games every 100 episodes).

In Fig. 3a, the 100-episode average of max Q grows from 0.03 (episode 100) to 5.57 (episode 2,000) and 12.14 (episode 5,000). It is still rising at the end. T5 (Table 3) checks the scale: on evaluation episodes, predicted Q exceeds the realized discounted return by 0.72 (6.9%).

In Fig. 3b, MA100 of the raw score grows from 275.0 (episode 100) to 600.8 (episode 1,000), 1,074.2 (episode 3,000) and 1,821.5 (episode 5,000).

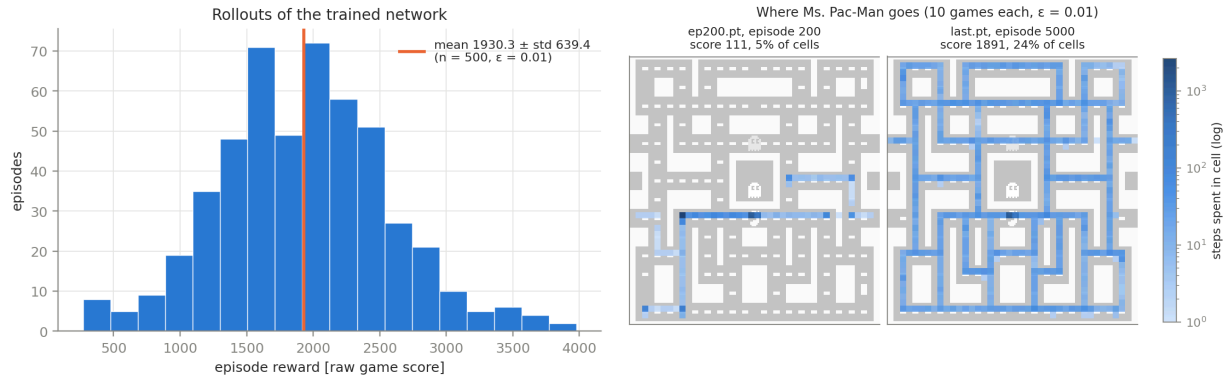


Figure 4. (a) 500 evaluation games of last.pt (ε = 0.01). (b) Ms. Pac-Man's position, located from her pixel color, over 10 games for the episode-200 snapshot and last.pt (episode 5,000), log color scale.

In Fig. 4a, last.pt scores 1,930.3 ± 639.4 over 500 games (min 270, max 3,980, 0 truncated). A uniform-random policy scores 260.1 ± 118.3 over 100 games, so the trained agent scores 7.4 times the random mean. last.pt is used instead of best.pt; see Table 2, last row.

In Fig. 4b, the episode-200 snapshot scores 111 and visits 5% of the maze cells. last.pt scores 1,891 and visits 24%. At episode 200 the network is nearly untrained, and at ε = 0.01 it repeats one route along a few corridors. That puts it below the random policy (111 vs 260.1); the greedy check at episode 200 also scored 110.

5. Edge cases

Table 2. Edge cases, how the code handles them, and the measured evidence.

Case	Handling	Evidence
Truncation vs termination	Target uses `terminated` only; truncated steps bootstrap	T3: bootstrap Q 20.30 vs 20; treating truncation as terminal gives unstable Q (23.58)
Episode-boundary frame stacking	Frames before an env reset are zero-filled; masking uses the reset flag, never `done`	T7: 844 of 159,872 sampled stacks would cross an episode; 0 after masking
Life loss (MsPacman)	Terminal for the target only; the game and frame stack continue	T7: 60 life-loss terminals, 30 game-over terminals in 30 games
Reward clipping cost	Learning uses sign(r); plots use the raw score	T7: 726 × 10, 1 × 50, 1 × 200, 1 × 400 points all become +1 (729), so a 400-point event counts the same as a pellet
int8 input scaling	Frames stored as int8; network input x/128 $\in [-1, 1)$	300,000 frames × 88 × 80 bytes ≈ 2.1 GB buffer
Stuck agent at eval	$\varepsilon = 0.01$ at evaluation, plus the v5 frame cap	0 of 500 eval games truncated; mean length 930.4 steps
Warmup	No update before `warmup` ≥ batch transitions	T6: first update at step 1,000, the CartPole warmup
Noisy checkpoint selection	Final policy chosen by a 100-game comparison, not the 5-game check	best.pt (check 2,366 at episode 2,600): 1,213.3 ± 829.9; last.pt: 1,900.6 ± 518.8 (100 games each)

6. Validation

Table 3. Validation tests. All pass (results/summary.json). Each prints one result line and writes one figure.

Test	Prediction	Measured	Command (run with python)
T1 γ sweep, 2 seeds	$Q \to 1/(1-\gamma) = 5, 10, 20$ (±10%); max length < 500 for $\gamma = 0.8$, = 500 for $\gamma \geq 0.9$	Q 4.88, 9.66, 18.10; max length 383/329, 500/500, 500/500	`validate/t1_gamma_sweep.py`
T2 10-state chain	max $\lvert Q - Q^* \rvert < 0.05$ vs value iteration; zero-reward control max $\lvert Q \rvert < 0.05$	0.0065; control 0.0005; greedy action right in every state	`validate/t2_chain_mdp.py`
T3 time limit 50	Bootstrap $Q \to 1/(1-\gamma) = 20$ (±10%); terminal variant unstable	Bootstrap 20.30; terminal 23.58, loss 18× higher	`validate/t3_truncation.py`
T4 ablation, 3 seeds	No target: peak Q > 20; full > no replay > no target	Peak Q 1.05×10^6; best check 447 > 349 > 36; final MA100 183 > 110 > 11	`validate/t4_ablation.py`
T5 Q vs return	bias = mean(Q − G), $G_t = \Sigma_k \gamma^k r_{t+k}$; $\lvert \text{bias} \rvert <$ 10% of mean G	CartPole −0.30 (−1.5%); MsPacman +0.72 (+6.9%), Pearson r 0.69	`validate/t5_q_vs_return.py`
T6 update trace	$y = r + \gamma(1 - \text{done}) \cdot Q_{\text{target}}(s', a^*)$ recomputes by hand; same seed twice gives identical runs	Max recompute error 2.2×10^{-8}; trace and 96-step greedy rollout identical	`validate/t6_trace.py`
T7 MsPacman edges	0 cross-episode stacks after masking; trained mean > random mean	844 → 0; 1,930.3 vs 260.1	`validate/t7_pacman_edges.py`

T1. Learned max Q is within 10% of $1/(1-\gamma)$ for all three γ: −2.4%, −3.4% and −9.5%. The horizon argument explains the length result. The Q gap between never failing and failing after 20 steps is $\gamma^{20}/(1-\gamma) = 0.058$, 1.22 and 7.17. At $\gamma = 0.8$ a late failure barely changes Q, so neither seed ever reaches 500 steps (maximum 383 and 329). Both seeds at $\gamma = 0.9$ and $\gamma = 0.95$ reach 500.

T3. With bootstrapping, Q converges to 20.30 (Fig. 5), 1.5% above $1/(1-\gamma) = 20$. Convergence speed follows $Q_K \approx 20(1 - 0.95^K)$, where K is the number of target syncs (one every 500 steps), so the run length was set to 1,500 episodes. The terminal variant is a negative control. Time is not in the state, so the same state is terminal at step 49 and not at step 0, and the target is non-Markov. Its Q reaches 23.58,

above 18.46 = $(1 - 0.95^{50})/(1 - 0.95)$, the largest return possible under the 50-step cap. Its loss over the last 500 episodes is 0.356 vs 0.019 (18×), and its reward MA100 falls to 10.9 after episode 300.

T4. Without a target network (sync every step), max Q diverges to 0.98-1.10×10^6 on all 3 seeds (Fig. 6b), against the bound of 20. Final MA100 is 11.3. Without replay (buffer = batch = 64), the mean best greedy check drops from 447 to 349 and the mean final MA100 from 183 to 110. The no-replay reward curve also dips between episodes 250 and 350 (Fig. 6a). The ordering full > no replay > no target holds for both measures.

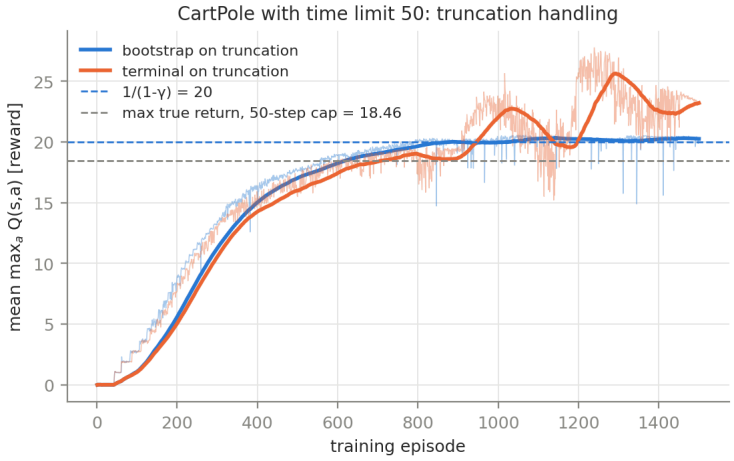


Figure 5. T3: max Q for bootstrapping vs terminal treatment of truncation, with 1/(1−γ) = 20 and the 50-step cap 18.46.

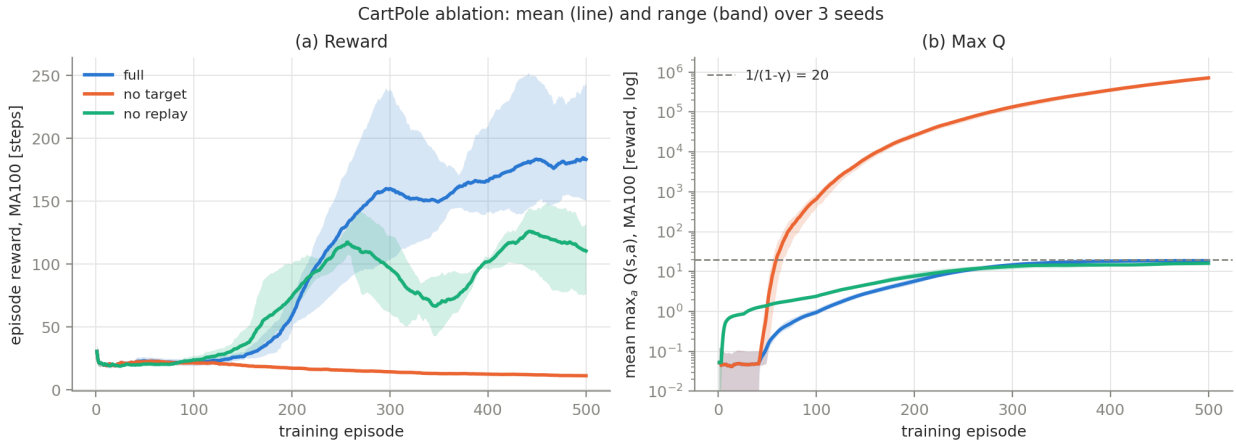


Figure 6. T4: (a) reward MA100, (b) max Q (log scale), mean and range over 3 seeds.

7. Given 3 more months

- Run 5+ seeds per configuration and report 95% confidence intervals instead of single runs.
- Ablate Rainbow components (prioritized replay, dueling, n-step, distributional, noisy nets) on MsPacman over 10M+ steps.
- Add time remaining to the state, which should make the T3 terminal variant Markov and stable.
- Track Q overestimation (T5 bias) across checkpoints with and without Double DQN.
- Sweep learning rate, target_sync and buffer size to measure how sensitive the final score is to each.